

2026년도 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드컴퓨팅
1. 각 모듈의 리소스는 해당 모듈 요구사항에 명시된 Region에 생성한다. 2. 웹 페이지 접근은 Chrome 또는 Firefox로 진행한다. 3. Shell 명령어의 출력은 AWS CLI 버전에 따라 일부 다를 수 있다. 4. 문제지와 채점지에 있는 <비번호>, <자율> 등은 선수 제출 값으로 확인한다. 5. 채점은 문항 순서대로 진행한다. 6. 부분 점수가 있는 문항은 채점 항목에 표시된 배점만 인정한다. 7. 부분 점수가 따로 없는 문항은 조건을 모두 만족해야 점수로 인정한다. 8. 리소스 정보를 읽어오는 항목은 CloudShell에서 자동화 채점 스크립트로 조회하고, 채점위원은 출력 결과를 기준으로 판정한다. 9. 선수 이의가 있는 경우 채점위원이 동일 명령을 직접 실행하여 확인할 수 있다. 10. IAM User Access Key를 별도로 생성하여 채점하지 않는다. 11. CloudShell을 실행한 IAM User 또는 Role 권한으로 채점한다. 12. 채점 시 대기가 필요한 경우 항목당 최대 3분을 초과하지 않는다.		

2. 채점기준표

1) 주요항목별 배점			직 종 명		클라우드컴퓨팅			
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제2과제	1	DocumentDB based NoSQL Application	7.5	0			0	1모듈
	2	Simplify Service Networking with VPC Lattice	7.5	0			0	2모듈
	3	Cloud Event Handling	7.5	0			0	3모듈
	4	Event-driven Pod Scaling with AWS SQS	7.5	0			0	4모듈
합계			30					

2) 채점방법 및 기준

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배 점
제2과제	1	Document based NoSQL Application	1	DocumentDB Cluster 및 Instance	1.5
			2	Secret 및 Client EC2	1.5
			3	Client Application 및 데이터 적재 상태	1.5
			4	DocumentDB Index 및 TTL 구성	1.5
			5	NoSQL 조회 기능 검증	1.5
	2	Simplify Service Networking with VPC Lattice	1	VPC	1.5
			2	Client/Service EC2 및 애플리케이션	1.5
			3	VPC Lattice Service Network 및 Service	1.5
			4	Target Group, Listener, Security Group	1.5
			5	End-to-End 기능 검증	1.5
	3	Cloud Event Handling	1	VPC, EC2, Security Group	1.5
			2	Security Group	1.5
			3	SNS Topic 및 Lambda	1.5
			4	CloudTrail, EventBridge Rule 및 Target	1.5
			5	최종 기능 검증	1.5
	4	Event-driven Pod Scaling with AWS SQS	1	EKS Cluster, VPC, Fargate Profile	1.2 5
			2	SQS Queue, IAM, SA	1.2 5
			3	KEDA/Karpenter	1.2 5
			4	Worker Application / KEDA ScaledObject	1.2 5
			5	Karpenter NodePool, EC2NodeClass, Worker EC2	1.2 5
			6	SQS 기반 Scale Out 및 최종 기능 검증	1.2 5

3) 채점 내용

순번	채점 항목
0	1) CloudShell에서 2과제 채점에 필요한 기본 도구를 확인합니다. 2) aws, curl, jq, kubectl 명령이 실행 가능한지 확인합니다. 3) kubectl이 없는 경우 아래 명령어로 EKS Cluster 버전에 맞는 kubectl을 설치 후 다시 확인합니다. 4) 본 항목은 채점 준비 절차이며 배점은 없습니다. <pre>aws --version curl --version jq --version if command -v kubectl >/dev/null 2>&1; then kubectl version --client else EKS_VERSION=\$(aws eks describe-cluster --region us-west-2 --name skills-sqs-cluster --query 'cluster.version' --output text) KUBECTL_VERSION="v\${EKS_VERSION}.0" curl -L -o /tmp/kubectl "https://dl.k8s.io/release/\${KUBECTL_VERSION}/bin/linux/amd64/kubectl" chmod +x /tmp/kubectl export PATH="/tmp:\$PATH" kubectl version --client fi</pre> ※ kubectl 설치 명령은 CloudShell에 kubectl이 없을 때만 실행합니다. ※ EKS Cluster가 아직 생성되지 않은 경우 4모듈 채점 직전에 본 설치 절차를 수행합니다. ※ 채점 스크립트는 kubectl을 자동 설치하지 않으므로, 4모듈 채점 전 본 준비 절차를 완료해야 합니다. 5) 아래 명령어를 입력하여 채점에 필요한 값을 변수로 선언합니다. 6) 값이 비어 있거나 None이면 해당 리소스가 요구 이름으로 생성되지 않은 것입니다. <pre>export NOSQL_CLIENT_EC2_PUBLIC_IP=\$(aws ec2 describe-instances --region ap-northeast-2 --filters Name=tag:Name,Values=skills-nosql-client-ec2 Name=instance-state-name,Values=running --query "Reservations[0].Instances[0].PublicIpAddress" --output text) export LATTICE_CLIENT_EC2_PUBLIC_IP=\$(aws ec2 describe-instances --region</pre>

	<pre> ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-ec2 Name=instance-state-name,Values=running --query "Reservations[0].Instances[0].PublicIpAddress" --output text) export SERVICE_NETWORK_ID=\$(aws vpc-lattice list-service-networks --region ap-northeast-1 --query "items[?name=='skills-lattice-sn'].id [0]" --output text) export TARGET_GROUP_ID=\$(aws vpc-lattice list-target-groups --region ap-northeast-1 --query "items[?name=='skills-lattice-order-tg'].id [0]" --output text) export SERVICE_ID=\$(aws vpc-lattice list-services --region ap-northeast-1 --query "items[?name=='skills-lattice-order-service'].id [0]" --output text) export SERVICE_EC2_SECURITY_GROUP_ID=\$(aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-service-ec2 Name=instance-state-name,Values=running --query "Reservations[0].Instances[0].SecurityGroups[0].GroupId" --output text) export QUEUE_URL=\$(aws sqs get-queue-url --region us-west-2 --queue-name skills-sqs-queue --query QueueUrl --output text) </pre>
1-1	1) 아래 명령어를 입력합니다. 2) DocumentDB Cluster/Instance와 KMS Key가 요구사항과 일치하는지 확인합니다. <pre> aws docdb describe-db-clusters --region ap-northeast-2 --db-cluster-identifier skills-nosql-docdb-cluster --output table aws docdb describe-db-instances --region ap-northeast-2 --db-instance-identifier skills-nosql-docdb-instance-1 --output table aws kms describe-key --region ap-northeast-2 --key-id alias/skills-nosql-docdb --output table </pre>
1-2	1) 아래 명령어를 입력합니다. 2) Secret에 username, password, host가 있고 Client EC2가 running 상태이며 Public IP가 있는지 확인합니다. <pre> aws secretsmanager get-secret-value --region ap-northeast-2 --secret-id skills-nosql-docdb-secret --query SecretString --output text aws ec2 describe-instances --region ap-northeast-2 --filters Name=tag:Name,Values=skills-nosql-client-ec2 Name=instance-state-name,Values=running --output table </pre>
1-3	1) 아래 명령어를 입력합니다. 2) /health, /v1/admin/summary가 HTTP 200을 반환하고 데이터 적재 상태가 요구사항과 일치하는지 확인합니다. <pre> curl -s -w "Wnhttp_code=%{http_code}Wn" http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/health curl -s -w "Wnhttp_code=%{http_code}Wn" http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/admin/summary </pre>
1-4	1) 아래 명령어를 입력합니다. 2) 컬렉션별 Index와 TTL 구성이 요구사항과 일치하는지 확인합니다. <pre> curl -s -w "Wnhttp_code=%{http_code}Wn" http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/admin/indexes </pre>

1-5	1) 아래 명령어를 입력합니다. 2) 각 조회 API가 HTTP 200을 반환하고 조건에 맞는 데이터가 포함되는지 확인합니다. <pre>curl -s -w "Wnhttp_code=%{http_code}Wn" http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/orders/0-1001 curl -s -w "Wnhttp_code=%{http_code}Wn" http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/customers/C001/orders curl -s -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/orders/pending?from=2026-06-01T00:00:00Z&to=2026-06-08T00:00:00Z" curl -s -w "Wnhttp_code=%{http_code}Wn" "http://\${NOSQL_CLIENT_EC2_PUBLIC_IP}:8080/v1/products/low-stock?warehouseId=W-A"</pre>
2-1	1) 아래 명령어를 입력합니다. 2) Client/Service VPC가 존재하고 CIDR이 각각 10.61.0.0/16, 10.62.0.0/16인지 확인합니다. <pre>aws ec2 describe-vpcs --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-vpc,skills-lattice-service-vpc --output table</pre>
2-2	1) 아래 명령어를 입력합니다. 2) Client/Service EC2 상태, Public IP 조건, Client /health HTTP 200 응답을 확인합니다. <pre>aws ec2 describe-instances --region ap-northeast-1 --filters Name=tag:Name,Values=skills-lattice-client-ec2,skills-lattice-service-ec2 Name=instance-state-name,Values=running --output table curl -s -w "Wnhttp_code=%{http_code}Wn" http://\${LATTICE_CLIENT_EC2_PUBLIC_IP}/health</pre>
2-3	1) 아래 명령어를 입력합니다. 2) Service Network가 존재하고, Service, VPC Association, Service Association이 ACTIVE 상태인지 확인합니다. <pre>aws vpc-lattice list-service-networks --region ap-northeast-1 --output table aws vpc-lattice list-services --region ap-northeast-1 --output table aws vpc-lattice list-service-network-vpc-associations --region ap-northeast-1 --service-network-identifier "\${SERVICE_NETWORK_ID}" --output table aws vpc-lattice get-service-network-vpc-association --region ap-northeast-1 --service-network-vpc-association-identifier "\${aws vpc-lattice list-service-network-vpc-associations --region ap-northeast-1 --service-network-identifier "\${SERVICE_NETWORK_ID}" --query "items[0].id" --output text}" --output table aws vpc-lattice list-service-network-service-associations --region ap-northeast-1 --service-network-identifier "\${SERVICE_NETWORK_ID}" --output table</pre>
2-4	1) 아래 명령어를 입력합니다. 2) Target Group, Target, Listener, Service EC2 Security Group 구성이 요구사항과 일치하는지 확인합니다. <pre>aws vpc-lattice list-target-groups --region ap-northeast-1 --output table</pre>

	<pre>aws vpc-lattice list-targets --region ap-northeast-1 --target-group-identifier "\$TARGET_GROUP_ID" --output table aws vpc-lattice list-listeners --region ap-northeast-1 --service-identifier "\$SERVICE_ID" --output table aws ec2 describe-security-groups --region ap-northeast-1 --group-ids "\$SERVICE_EC2_SECURITY_GROUP_ID" --output json</pre>
2-5	1) 아래 명령어를 입력합니다. 2) Client API가 HTTP 200을 반환하고 order_id=1001, via=vpc-lattice 값이 포함되는지 확인합니다. <pre>curl -s -w "Wnhttp_code=%{http_code}Wn" "http://\${LATTICE_CLIENT_EC2_PUBLIC_IP}/v1/client/orders?id=1001"</pre>
3-1	1) 아래 명령어를 입력합니다. 2) VPC, EC2, 보호 대상 Security Group이 존재하고 연결 상태가 요구사항과 일치하는지 확인합니다. <pre>aws ec2 describe-vpcs --region ap-southeast-1 --filters Name=tag:Name,Values=skills-ceh-vpc --output table aws ec2 describe-instances --region ap-southeast-1 --filters Name=tag:Name,Values=skills-ceh-ec2 Name=instance-state-name,Values=running --output table aws ec2 describe-security-groups --region ap-southeast-1 --filters Name=tag:Name,Values=skills-ceh-protected-sg --output table</pre>
3-2	1) 아래 명령어를 입력합니다. 2) skills-ceh-protected-sg의 Inbound 규칙이 0개인지 확인합니다. <pre>aws ec2 describe-security-groups --region ap-southeast-1 --filters Name=tag:Name,Values=skills-ceh-protected-sg --query "SecurityGroups[].IpPermissions" --output json</pre>
3-3	1) 아래 명령어를 입력합니다. 2) SNS Topic과 Lambda Runtime, Handler, Timeout, Environment가 요구사항과 일치하는지 확인합니다. <pre>aws sns list-topics --region ap-southeast-1 --output table aws lambda get-function-configuration --region ap-southeast-1 --function-name skills-ceh-remediate-fn --output table</pre>
3-4	1) 아래 명령어를 입력합니다. 2) CloudTrail Trail, EventBridge Rule Event Pattern, Target, Lambda Resource Policy가 요구사항과 일치하는지 확인합니다. <pre>aws cloudtrail get-trail-status --region ap-southeast-1 --name skills-ceh-cloudtrail --output table aws events describe-rule --region ap-southeast-1 --name skills-ceh-sg-change-rule --event-bus-name default --output json aws events list-targets-by-rule --region ap-southeast-1 --rule skills-ceh-sg-change-rule --event-bus-name default --output table aws lambda get-policy --region ap-southeast-1 --function-name skills-ceh-remediate-fn --query Policy --output text</pre>

3-5	1) 아래 명령어로 테스트용 Inbound 규칙을 추가합니다. 2) 보호 대상 Security Group ID를 변수로 선언하고, CloudTrail 이벤트와 동일하게 groupId를 포함한 Payload로 Lambda를 직접 호출합니다. 3) 180초 이내 Inbound 규칙이 다시 0개가 되고 Lambda Log Group이 생성되는지 확인합니다. <pre> export PROTECTED_SECURITY_GROUP_ID=\$(aws ec2 describe-security-groups --region ap-southeast-1 --filters Name=tag:Name,Values=skills-ceh-protected-sg --query "SecurityGroups[0].GroupId" --output text) aws ec2 authorize-security-group-ingress --region ap-southeast-1 --group-id "\$PROTECTED_SECURITY_GROUP_ID" --protocol tcp --port 22 --cidr 0.0.0.0/0 jq -n W --arg sg "\$PROTECTED_SECURITY_GROUP_ID" W '{{detail:{eventName:"AuthorizeSecurityGroupIngress",requestParameters:{groupId:\$sg}}}}' W > /tmp/skills-ceh-remediate-event.json aws lambda invoke --region ap-southeast-1 --function-name skills-ceh-remediate-fn --cli-binary-format raw-in-base64-out --payload file:///tmp/skills-ceh-remediate-event.json /tmp/skills-ceh-remediate-output.json aws ec2 describe-security-groups --region ap-southeast-1 --group-ids "\$PROTECTED_SECURITY_GROUP_ID" --query "SecurityGroups[0].IpPermissions" --output json aws logs describe-log-groups --region ap-southeast-1 --log-group-name-prefix /aws/lambda/skills-ceh-remediate-fn --output table </pre>
4-1	1) 아래 명령어를 입력합니다. 2) EKS Cluster와 Fargate Profile 2개가 ACTIVE이며 CloudShell에서 kubectl 접근 가능한지 확인합니다. <pre> aws eks describe-cluster --region us-west-2 --name skills-sqs-cluster --output table aws eks describe-fargate-profile --region us-west-2 --cluster-name skills-sqs-cluster --fargate-profile-name skills-sqs-fp-keda --output table aws eks describe-fargate-profile --region us-west-2 --cluster-name skills-sqs-cluster --fargate-profile-name skills-sqs-fp-karpenter --output table aws eks update-kubeconfig --region us-west-2 --name skills-sqs-cluster kubectl get nodes -l eks.amazonaws.com/compute-type=fargate -o wide </pre>
4-2	1) 아래 명령어를 입력합니다. 2) SQS Queue와 ServiceAccount IRSA annotation이 요구사항과 일치하는지 확인합니다. <pre> aws sqs get-queue-url --region us-west-2 --queue-name skills-sqs-queue aws sqs get-queue-attributes --region us-west-2 --queue-url "\$QUEUE_URL" --attribute-names QueueArn VisibilityTimeout --output table kubectl get serviceaccount keda-operator -n keda -o yaml kubectl get serviceaccount karpenter -n karpenter -o yaml kubectl get serviceaccount sqs-worker-sa -n skills-sqs -o yaml </pre>

4-3	1) 아래 명령어를 입력합니다. 2) KEDA/Karpenter Controller Pod가 Running 상태이며 Fargate Node에서 실행되는지 확인합니다. <pre>kubectl get deployment,pod -n keda -o wide</pre> <pre>kubectl get deployment,pod -n karpenter -o wide</pre>
4-4	1) 아래 명령어를 입력합니다. 2) Worker Deployment, 환경변수, Node Selector, ScaledObject, TriggerAuthentication 구성이 요구사항과 일치하는지 확인합니다. <pre>kubectl get deployment sqs-worker -n skills-sqs -o yaml</pre> <pre>kubectl get scaledobject sqs-worker-scaledobject -n skills-sqs -o yaml</pre> <pre>kubectl get triggerauthentication sqs-worker-trigger-auth -n skills-sqs -o yaml</pre>
4-5	1) 아래 명령어를 입력합니다. 2) NodePool, EC2NodeClass, Worker EC2 Node, Worker Pod 배치가 요구사항과 일치하는지 확인합니다. <pre>kubectl get nodepool skills-sqs-nodepool -o yaml</pre> <pre>kubectl get ec2nodeclass skills-sqs-nodeclass -o yaml</pre> <pre>kubectl get nodes -l</pre> <pre>karpenter.sh/nodepool=skills-sqs-nodepool,skills-nodepool=event-worker -o wide</pre> <pre>kubectl get pods -n skills-sqs -l app=sqs-worker -o wide</pre>
4-6	1) 아래 명령어로 SQS 메시지를 12개 발행합니다. 2) 180초 이내 Worker Pod와 Karpenter EC2 Worker Node가 증가하고 Queue depth가 감소하는지 확인합니다. <pre>for i in \$(seq 1 12); do aws sqs send-message --region us-west-2 --queue-url</pre> <pre>"\$QUEUE_URL" --message-body "judge-\$i"; done</pre> <pre>aws sqs get-queue-attributes --region us-west-2 --queue-url "\$QUEUE_URL"</pre> <pre>--attribute-names ApproximateNumberOfMessages</pre> <pre>ApproximateNumberOfMessagesNotVisible --output table</pre> <pre>kubectl get pods -n skills-sqs -l app=sqs-worker -o wide</pre> <pre>kubectl get nodes -l</pre> <pre>karpenter.sh/nodepool=skills-sqs-nodepool,skills-nodepool=event-worker -o wide</pre>